

Note de cadrage Refonte plateforme.

Décisions verrouillées, architecture, modèle de données, roadmap, hébergement, stratégie Odoo, état des maquettes.

Version **v3** · 28 mai 2026 · **Topias1/dloazur**

Version : v3 (remplace v2 du 2026-05-27) **Date :** 2026-05-28 **Statut :** **maquettes v1 livrées** (six écrans HTML/Tailwind in-repo, palette extraite des supports imprimés réels), prêtes pour validation client. En attente : (1) retour Pierre sur l'aperçu v1, (2) réponses Odoo avant Phase 1a, (3) hébergement public de l'aperçu (GitHub Pages).

Changements vs v2 :

- §2 « Maquettage » : Claude Design abandonné ; remplacé par le skill **impeccable** in-repo. Les maquettes vivent dans `mockups/v1/`, **versionnées par dossier** pour gérer les itérations clients (`v1/` , `v2/` ...), transposables directement en Blade/Livewire.
- §2 « Palette » : tokens **extraits des supports imprimés réels** (logo vectoriel, carte de visite, flyer, plaquette hospitalité) en OKLCH. Azur exact `#0080ff` (logo), marine `#154c79` (fond carte de visite), lagon `#2fb8c8` (texte de carte), soleil en accent. Typographie : **Fredoka** (titres, rondeur qui répond au mot-logo « AZUR ») + **Inter** (corps).
- §9 « Maquettes & Design » : entièrement réécrite. Six écrans construits, design system documenté dans `DESIGN.md` (format Stitch + tokens OKLCH + composants) + `PRODUCT.md` (stratégie, registre, anti-références). Trend critique impeccable : **27/40 → 31/40**.
- §13 Prochaines étapes : focus déplacé sur la validation Pierre + l'hébergement public de l'aperçu.

1. Contexte

Dlo Azur Piscines (Martinique) — entretien/nettoyage de piscines & spas, **opérateur seul**. Pas de construction ni vente.

- **Existant** : site vitrine sur **Zyro** (builder Hostinger) — accueil, services, réalisations, blog, formulaire contact, réseaux. **Aucune fonctionnalité métier, pas d'espace client**.
- **Contact** : +596 696 94 00 54 · contact@dloazurpiscines.com
- **Demande initiale** : « refaire le site » + 2 lots métier (suivi des passages + facturation Odoo ; diagnostic piscine commercialisable).

À propos de la maquette `diagnostic-dloazur.html` : c'est une app **React compilée** (build single-file) qui prototypait déjà le diagnostic *et* un squelette d'espace client (`/api/me/visits` , `/api/clients` , auth cookie). **Le client n'en est pas satisfait → maquette jetée**. On en conserve uniquement **la logique métier du diagnostic** (arbre de décision, formules de doses, libellés de traitement) comme **spécification** pour réécrire propre (Phase 2).

2. Décisions verrouillées

Décision	Choix	Raison
Stack	Laravel 11 + Livewire + Alpine.js + Tailwind + PostgreSQL	Fluence PHP du dev ; app CRUD/portail/facturation/SEO « batteries incluses » ; maintenance solo durable (on ne parie pas sur un assistant IA toujours dispo).
Front	Server-rendered (Livewire/Blade). Pas de React, pas de SPA.	Cohérent avec le stack ; React abandonné.
Architecture	Monolithe unique : vitrine + portail + admin + diagnostic	Un seul codebase, un seul déploiement.
Multi-tenant	Non (single-tenant)	Lot 2 ciblé A+B, pas de marque blanche (C) pour l'instant.
Offline	Offline-first dès le MVP , sur l'écran de saisie d'un passage uniquement	Réseau « hasardeux » en Martinique + photos systématiques → la saisie doit survivre sans réseau.
Hébergement	Laravel Cloud, région EU (Francfort)	Managé max, scale-to-zero, Postgres managé inclus, ~4-7 €/mois.
Stockage photos	Scaleway Object Storage (Paris)	Hébergeur européen, RGPD propre, coût négligeable.
Maquettage	impeccable (skill in-repo, Claude Code)	Six écrans HTML/Tailwind statiques dans mockups/v1/ , transposables directement en Blade. Versionnés par dossier (v1/ , v2/ ...) pour les itérations clients. Pas de Figma, pas de Claude Design.
Palette visuelle	Extraite des supports imprimés réels. Azur #0080ff (logo) + marine #154c79 (carte de visite) + lagon #2fb8c8 + soleil. Fredoka + Inter. Tout en OKLCH.	Pas de couleurs inventées : sampling du logo vectoriel et de la carte de visite. Détaillée dans DESIGN.md (format Stitch) et 2026-05-27-dloazur-design-system.md (v3).

3. Architecture

Monolithe Laravel découpé en modules. **Point clé** : Livewire fait un aller-retour serveur à chaque interaction → **inutilisable hors-ligne**. L'archi se dédouble donc proprement :

- **Online (Livewire / Blade)** — vitrine, dashboards pro, historique, espace client, facturation, diagnostic.
- **Offline-first (Alpine + JS + IndexedDB + Service Worker)** — **uniquement l'écran de saisie d'un passage** : persistance locale, file d'attente d'upload photos, synchronisation au retour réseau. Seule brique PWA « dure ».

Module	Rôle	Accès
Vitrine	Pages marketing SEO local	Public
Auth & rôles	Pro (email + mot de passe) / Client (magic link)	—
Suivi passages	Saisie mobile offline-first + historique	Pro
Portail client	Lecture seule : passages, mesures, photos	Client
Facturation	Catalogue, contrats, factures, pont Odoo	Pro
Diagnostic	Wizard + calcul de doses, paiement Stripe	Public / freemium

Convention : code & tables en anglais (idiomatique Laravel), **UI en français**.

4. Modèle de données

Phase 0 (MVP suivi)

- **users** — `id, name, email, password (nullable), role enum(pro|client), client_id (FK-clients, nullable), timestamps` *Le pro = compte staff ; un client se connecte par magic link et pointe vers son enregistrement `clients`.*
- **clients** — `id, name, email, phone, address, notes, timestamps`
- **pools** — `id, client_id (FK), label, volume_m3, type enum(liner|coque|béton|autre), filtration enum(sable|verre|cartouche|poche), has_electrolyseur bool, equipment json (nullable), notes, timestamps`
- **visits** — `id, pool_id (FK), performed_at, ph, free_chlorine, tac, salt_ppm (nullable), water_temp (nullable), actions text, notes text, client_uuid (idempotence offline), created_by (FK-users), timestamps`
- **visit_photos** — `id, visit_id (FK), disk_path, original_name, uploaded_at` *(ou `spatie/laravel-medialibrary`)*

Relations : `Client 1-* Pool` · `Pool 1-* Visit` · `Visit 1-* VisitPhoto` · `User(client) *-1 Client` . **Idempotence offline :** chaque passage reçoit un `client_uuid` généré côté téléphone → la re-synchro ne crée pas de doublon.

Phase 1a (ajouts)

- **products** — catalogue facturable (`name, type service|produit, unit_price, vat_rate, odoo_product_id nullable`)
- **contracts** — `client_id, type enum(ponctuel|forfait_mensuel|forfait_saisonnier), price, start_at, end_at, status`
- **invoices** — `client_id, visit_id?, contract_id?, number, status enum(draft|sent|paid), total, pdf_path, odoo_invoice_id?, issued_at, paid_at`
- **invoice_lines** — `invoice_id, product_id, label, qty, unit_price, vat_rate`
- **signatures** — image de signature liée au passage (`visit_id, image_path, signed_at`)

Phase 2 (ajouts)

- **diagnostics** — `user_id (nullable, anonyme permis), inputs json, decision_path json, recommendations json, created_at`
 - **subscriptions** — via `laravel/cashier` (Stripe)
 - Contenu du diagnostic (arbre + doses) = données de config/seed extraites de la maquette.
-

5. Roadmap

Principe : *chaque phase est utilisable en production sans la suivante.*

Phase V – Vitrine

Refonte de la vitrine dans le codebase Laravel, **remplace Zyro**. Pages : accueil, services, réalisations (galerie), blog, contact. SEO local Martinique, mobile-first, formulaire contact, liens réseaux/WhatsApp, CTA avis Google. Reprise du contenu/photos existants.

Phase 0 – MVP Suivi (offline-first)

On fait : auth (pro mdp / client magic link) · clients + piscine (mono-piscine en UI) · **saisie d'un passage hors-ligne** (mesures pH/chlore/TAC/sel, actions, notes, **photos**) avec synchro au retour réseau · historique pro (filtres client/date) · espace client lecture seule. **On ne fait pas** : facturation, Odoo, signature, planning, notifications métier (hors e-mail d'auth magic link), app native.

Phase 1a – Facturation & Odoo

POC Odoo en tout premier (2-3 j) pour valider la viabilité selon sa version (voir §6). Puis : catalogue produits/services · contrats (ponctuel + forfait) · génération de facture à la clôture d'un passage → **Odoo (API) ou export CSV** selon le résultat du POC · récupération du statut de paiement · **compte-rendu PDF auto** · **signature client** sur le téléphone du pro.

Phase 1b – Notifications (*réduit*)

Email compte-rendu après passage + rappel passage J-1, **option WhatsApp**. (*Les rôles techniciens de la note v0.2 sont supprimés : le client travaille seul.*)

Phase 2 – Diagnostic (commercialisation)

Réécriture propre du wizard « ma piscine est verte » : diagnostic enrichi, **calculs de doses selon le bassin**, plan d'action chiffré, suivi multi-mesures, **paiement Stripe**. **Monétisation** : **pistes A + B** — vente directe particuliers (abonnement Stripe) **et** module premium en upsell sur ses clients d'entretien. **Piste C (marque blanche multi-tenant) écartée pour l'instant.**

6. Stratégie Odoo (critique pour 1a)

Fait vérifié (doc officielle Odoo) : « *Access to data via the external API is only available on Custom Odoo pricing plans. Access to the external API is not available on One App Free or Standard plans.* » L'API externe (XML-RPC) exige donc le plan **Custom (29,90 €/user/mois)** sur Odoo Online ; **Standard (19,90 €)** et **One App Free** ne l'ont pas. Odoo **Community auto-hébergé** a l'API gratuitement mais ajoute de l'ops (va contre « managé/simple »).

Approche : POC en début de Phase 1a pour trancher entre deux voies : - **Voie A — API live (XML-RPC via package PHP)** si le client est sur **Custom** ou **Community/Enterprise** avec API : push facture (draft ou émise) + pull statut de paiement, temps réel. - **Voie B — Pont sans API (CSV)** si le client reste sur Free/Standard : l'app génère les factures (source de vérité) et exporte en CSV → import manuel dans Odoo. Zéro surcoût Odoo.

Source de vérité par défaut : l'app (à confirmer avec lui — voir §11).

7. Hébergement & RGPD

- **Laravel Cloud** (GA) — région **EU / Francfort**. Plan Starter 0 €/mois + à l'usage, **scale-to-zero** (hibernation), **Postgres managé (Neon)** inclus. Coût réel attendu **~4-7 €/mois** vu le volume.
 - **Nuance RGPD** : Laravel Cloud tourne sur **AWS** (maison-mère US) → post-Schrems II, transferts couverts par **SCCs** (fournies par AWS). Acceptable ici car données **peu sensibles** (noms, adresses, photos de piscines — pas de santé/finance). Si besoin d'un récit RGPD « zéro paperasse » plus tard : bascule vers hébergeur **européen** (Scaleway Paris / Hetzner DE) + Ploi.
 - **Photos → Scaleway Object Storage (Paris)** : européen, RGPD propre, ~0,01 €/Go/mois.
-

8. Offline-first – approche technique

- **Périmètre minimal** : seul l'écran de saisie d'un passage est offline-first. Le reste (historique, dashboards) suppose le réseau.
 - **Mécanique** : Service Worker (PWA installable sur smartphone) + **IndexedDB** pour stocker passage + photos en local + **file d'attente de synchro** rejouée au retour réseau.
 - **Idempotence** : `client_uuid` par passage → pas de doublon à la re-synchro.
 - **Conflits** : quasi inexistants (un seul opérateur, données en création seule) → pas de résolution de conflit complexe nécessaire.
 - **Implémentation** : Alpine.js + JS vanilla pour cette brique (**pas Livewire**, qui exige le réseau).
 - **Risque #1 à dérisquer tôt** : quotas IndexedDB et comportement du Service Worker sur **iOS Safari**. Tester sur le smartphone réel du client avant d'engager le module.
-

9. Maquettes & Design

Outil retenu : le skill **impeccable** (Anthropic Labs, in-repo) opéré dans Claude Code. Conversationnel, design system in-process, écrit du HTML+Tailwind statique directement portable en Blade/Livewire. Claude Design (claude.ai/design) a été évalué puis abandonné : in-repo gagne sur traçabilité git, versioning, et coût d'itération.

Pourquoi pas Figma : pas de bénéfice ici (dev solo, pas de designer dans l'équipe). impeccable produit directement des maquettes implémentables sur le stack cible, et garde tout en git.

Source de vérité visuelle

Fichier	Rôle
<code>PRODUCT.md</code> (racine repo)	Stratégie : registre, utilisateurs, raison d'être, personnalité, anti-références, 5 principes, a11y.
<code>DESIGN.md</code> (racine repo)	Système visuel format Stitch : YAML frontmatter (tokens OKLCH, typographie, composants) + 6 sections (Overview, Colors, Typography, Elevation, Components, Do's and Don'ts). North Star : « L'artisan du lagon ».
<code>.impeccable/design.json</code>	Sidecar : rampes tonales, ombres, motion, breakpoints, 9 primitives de composants rendues (HTML+CSS self-contained).
<code>2026-05-27-dloazur-design-system.md</code>	Doc historique (v3) : récapitulatif palette, typo, motif, écrans livrés, portage Laravel.

Versioning des maquettes

Les maquettes vivent dans `mockups/v<N>/` pour faciliter les itérations après retours client. Le retour de Pierre sur la v1 produit une v2 ; la v1 reste consultable et comparable.

```
mockups/
index.html      # redirige vers la version courante
v1/
  index.html    # galerie des 6 écrans
  vitrine.html  passage.html  portail.html
  dashboard.html auth.html    styleguide.html
  app.css       theme.js
  previews/*.png # thumbnails de la galerie
```

Écrans livrés (v1 – 28 mai 2026)

#	Écran	Univers (registre)	État
1	<code>vitrine.html</code>	brand (public)	hero authentique, services asymétriques, offre hospitalité B2B, réalisations, Pierre, teaser espace client, témoignages, CTA, footer + QR
2	<code>passage.html</code>	product (terrain)	saisie offline-first dans un phone-frame : header, bandeau hors-ligne ambre, mesures 2x2 + steppers, actions, photos avec statut, notes, save-bar collante
3	<code>dashboard.html</code>	product (pro)	sidebar desktop / bottom-nav mobile, KPI, tournée du jour, à recontacter, derniers comptes-rendus
4	<code>portail.html</code>	product (client)	piscine, dernier passage, mot de Pierre, photos, historique, lien magique
5	<code>auth.html</code>	product (entrée)	écran scindé marine/sable, bascule pro / client (magic link)
6	<code>styleguide.html</code>	fondations	tokens, palette, typo, motif, composants — design system rendu

Index galerie : `mockups/v1/index.html` . Ouvrable via `python3 -m http.server` à la racine du dépôt, puis `http://localhost:8000/mockups/v1/` .

Critique & polish (impeccable)

La vitrine a été passée à `/impeccable critique` puis `/impeccable polish` , snapshots persistés dans `.impeccable/critique/` .

Run	Score	P1	Détail
Initial	27 / 40	2	hero-metric template, 4 em-dashes, kicker repetition
Post-fix	31 / 40	0	tous P1/P2 résolus ; polish ajouté (OG meta, skip-link, lazy loading)

Détecteur déterministe : 0 errors, 0 warns sur les 6 écrans. Le détecteur upstream du skill manquait son bundle ; patch local dans [tools/impeccable-detector/](#) (em-dash, side-stripe border, gradient text, glassmorphism, hero-metric, kicker repetition, identical card grids).

Portage Laravel

- Reprendre les tokens de [mockups/v1/theme.js](#) dans le [tailwind.config.js](#) du projet Laravel (mêmes valeurs OKLCH).
 - [app.css](#) → fichier CSS d'app (fonts via Google Fonts ou [@fontsource](#) , variables, [.ripple](#) , motif vague).
 - Vitrine / dashboard / portail → Blade + Livewire.
 - [passage](#) reste hors Livewire : Alpine + IndexedDB + Service Worker, conforme à la contrainte offline-first §8.
 - Icônes : jeux SVG inline (style Lucide) déjà présents dans les maquettes ; glyphe WhatsApp officiel (Simple Icons).
-

10. Coûts indicatifs (récurrents)

Poste	Coût
Hébergement Laravel Cloud EU	~4-7 €/mois
Stockage photos Scaleway	quelques centimes/mois
Odoo — si Voie A (plan Custom, 1 user)	+29,90 €/mois
Odoo — si Voie B (CSV)	0 €
Stripe (Phase 2)	~1,5 % + 0,25 €/transaction (cartes EU)
Domaine	déjà possédé

11. Questions restantes pour le client (bloquent Phase 1a)

- **Version exacte d'Odoo** : Community / Enterprise / Online (quel plan) / Odoo.sh ? *(détermine Voie A vs B)*
- **Prêt à fournir des credentials API techniques** (clé API) ?
- Factures **en draft** (validation manuelle) ou **émises directement** ?
- **Source de vérité** des contacts : l'app ou Odoo ?
- **Signature client** : systématique à chaque passage, ou ponctuelle ?

(Déjà répondu : ~10 passages/semaine, ~dizaine de clients à l'année · travaille seul · facturation mix ponctuel/forfait 50-50 · smartphone uniquement · réseau hasardeux · photos systématiques · 1 piscine/client · Lot 2 cible A+B, pas C.)

12. Hypothèses & risques

- **H1** : Odoos probablement sur un plan **sans API** → prévoir la Voie B (CSV) comme défaut tant que le POC n'a pas tranché. **Ne pas découvrir au jour 3.**
 - **H2** : L'offline-first alourdit le MVP (Phase 0) — assumé, car un MVP online-only serait inutilisable sur le terrain.
 - **H3** : Contenu/photos de la vitrine récupérables depuis le site Zyro actuel (à confirmer, sinon shooting).
 - **H4** : RGPD — données peu sensibles, AWS+SCCs acceptable ; réévaluer si le périmètre de données change.
 - **H5** : IndexedDB + Service Worker sur iOS Safari — quotas et restrictions variables. POC à faire tôt sur le smartphone réel du client.
 - **H6** : Extraire l'arbre de décision et les formules de doses de la maquette React **avant de la jeter** — sinon spec perdue pour la Phase 2.
-

13. Prochaines étapes

#	Étape	État
1	Note de cadrage v3 (cette note) à jour	✅ fait
2	Maquettes v1 (six écrans) construites et critique-validées (27 → 31 / 40)	✅ fait
3	PRODUCT.md + DESIGN.md (format Stitch) en place	✅ fait
4	Hébergement public de l'aperçu v1 sur GitHub Pages (repo public)	🕒 à faire
5	PDF récap client pour Pierre (docs/exports/recap-client.pdf)	🕒 à faire
6	Retour Pierre sur la v1 → décisions visuelles, naissance de la v2	🕒 en attente client
7	Extraction spec diagnostic depuis diagnostic-dloazur.html (arbre de décision, formules) avant que la maquette React jetée ne soit oubliée	🕒 à faire avant Phase 2
8	Réponses Odoo du client (§11)	🕒 en attente client
9	Phase V — Vitrine Laravel : initialiser le projet Laravel 11, transposer mockups/v<N>/vitrine.html en Blade, déployer sur Laravel Cloud EU, basculer le DNS depuis Hostinger	🕒 à faire après v2 validée
10	Phase 0 — MVP suivi : modèle de données §4, écran de saisie offline-first (Alpine + IndexedDB + Service Worker), historique pro, portail client magic link	🕒 après Phase V

Sources vérifiées : [Odoo — External API \(restriction plan Custom\)](#) · [Odoo Pricing](#) · [Laravel Cloud Pricing](#) · [Laravel Cloud — régions EU & nuance RGPD/Schrems II](#) · [Best Laravel hosting 2026](#) · [Repo GitHub](#) [Topias1/dloazur](#)